```
$ kubectl get all -l app=catalog-service

NAME                                    READY   STATUS    RESTARTS   AGE
pod/catalog-service-68bc5659b8-k6dpb    1/1     Running   0          42s

NAME                                READY   UP-TO-DATE   AVAILABLE   AGE
deployment.apps/catalog-service     1/1     1            1           42s

NAME                                           DESIRED   CURRENT   READY   AGE
replicaset.apps/catalog-service-68bc5659b8     1         1         1       42s
```

Since you used labels consistently in your Deployment manifest, you can use the label `app=catalog-service` to fetch all the Kubernetes objects related to the Catalog Service deployment. As you can see, the declaration in deployment.yml resulted in the creation of a Deployment, a ReplicaSet, and a Pod.

To verify that Catalog Service started up correctly, you can check the logs from its Deployment as follows:

```
$ kubectl logs deployment/catalog-service
```

NOTE You can monitor whether Pods have been created successfully by inspecting the `STATUS` column when you run `kubectl get pods`. Should a Pod fail to be deployed, check that column. Common error statuses are `ErrImagePull` or `ImagePullBackOff`. They happen when Kubernetes can't pull the image used by the Pod from the configured container registry. We're currently working with local images, so make sure you built and loaded a Catalog Service container image into minikube. You can use the `kubectl describe pod <pod_name>` command to get more information about the error and `kubectl logs <pod_name>` to get the application logs from a specific Pod instance.

When deploying containers in a cloud environment like a Kubernetes cluster, you'll want to be sure it has enough resources to operate. In chapter 15, you'll learn how to assign CPU and memory resources to a container running in Kubernetes and how to configure memory for the JVM by applying the Java memory calculator provided by Cloud Native Buildpacks. For now, we'll rely on the default resource configuration.